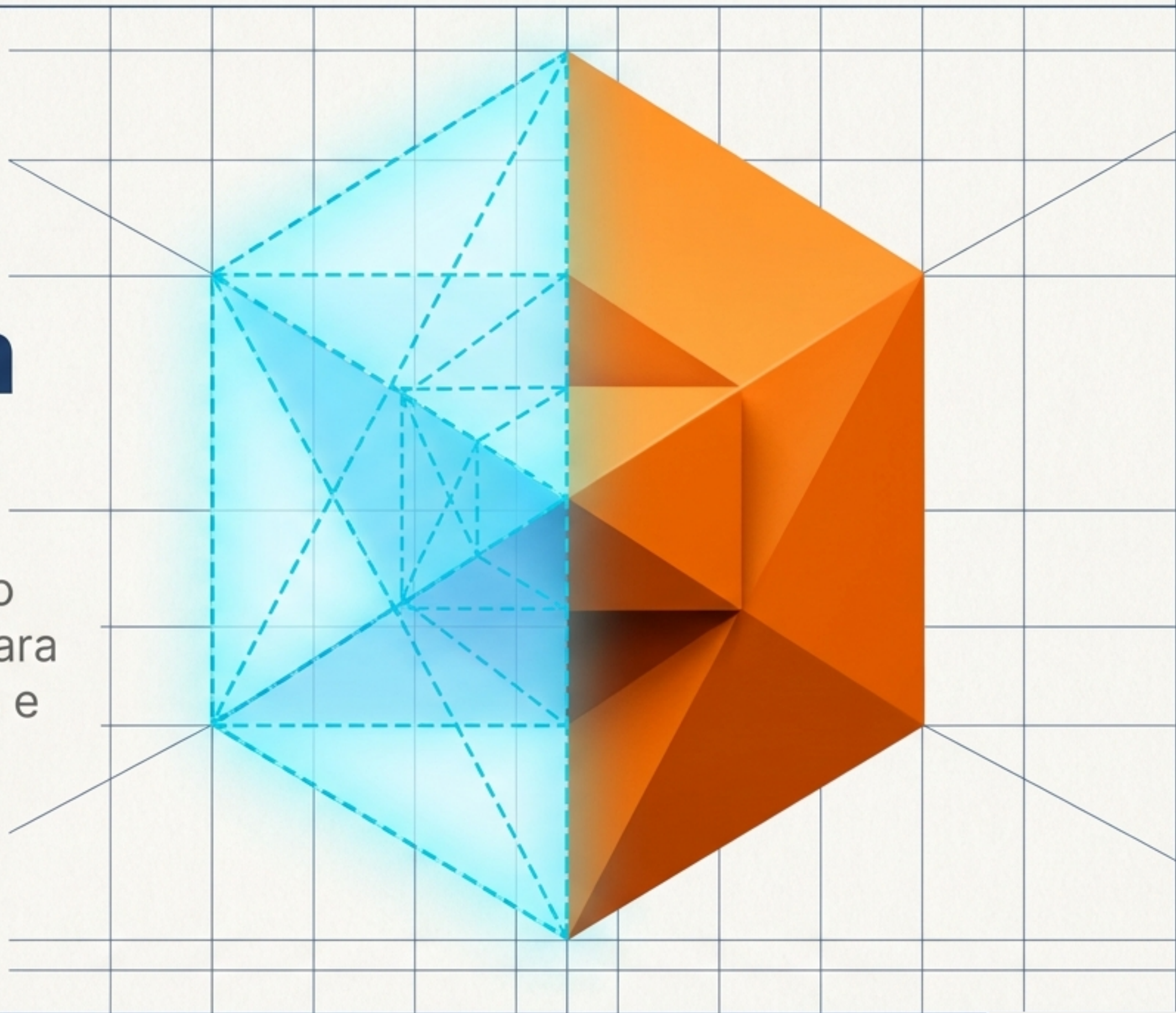


# Classes Abstratas em Python

A Arquitetura do Código: Usando Abstract Base Classes (ABCs) para definir contratos, forçar padrões e construir sistemas escaláveis.





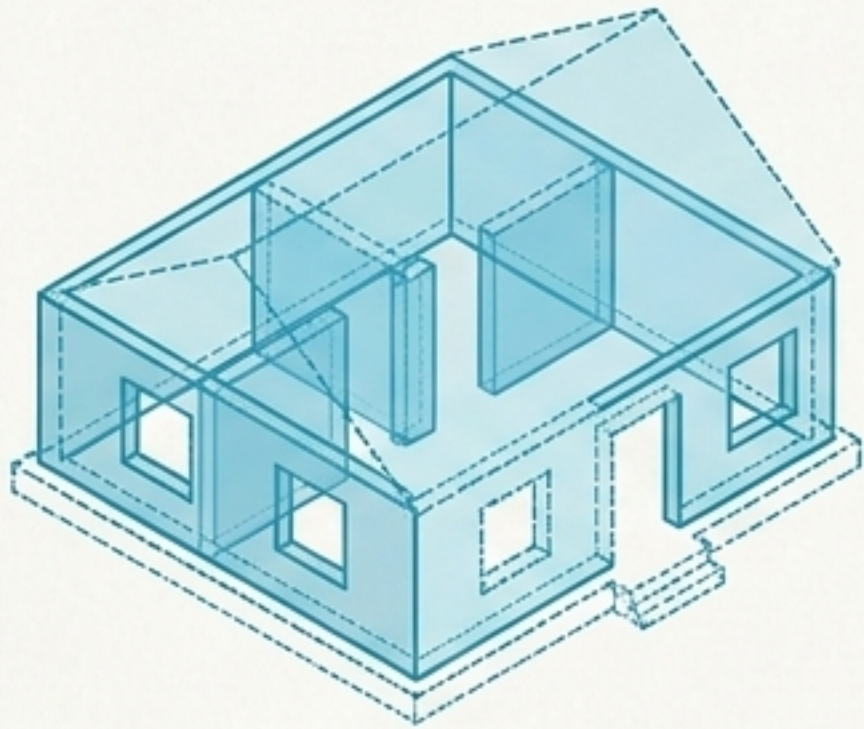
# Sumário da Apresentação





# O Conceito Fundamental

## Classe Abstrata (Abstract Base Class)



Define a estrutura, os métodos e os elementos obrigatórios. Possui lacunas intencionais. Você não pode morar nela (não instanciável).

## Classe Concreta (Concrete Class)

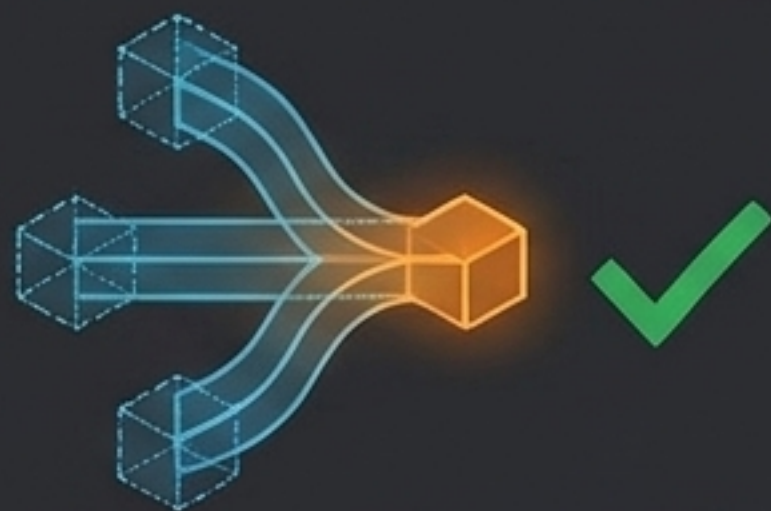


Preenche todas as lacunas deixadas pela planta. É um objeto funcional, finalizado e habitável (instanciável).



# Por que utilizar Abstract Base Classes?

Agrupamento de comportamento e garantia de execução.



## Reúso de Código

Quando você deseja especificar dados ou comportamentos comuns a um conjunto de classes, mas essas peças não são suficientes para formar uma instância isolada e funcional.



## Forçar Comportamentos (Contratos)

Quando você quer garantir que qualquer classe herdeira implemente métodos específicos (forçando nomenclaturas e parâmetros corretos), ideal para criadores de bibliotecas e APIs.



# O Erro de Contrato: Quando a regra é quebrada

```
from collections import MutableSequence
```

```
class Bag(MutableSequence):
```

```
    pass
```

A classe herda de uma ABC built-in (MutableSequence), mas usa pass ignorando todos os métodos abstratos obrigatórios.

```
bag = Bag()
```

`TypeError: Can't instantiate abstract class Bag with abstract methods __delitem__, __getitem__, __len__, __setitem__, insert`

O Python não avisa no import; ele apenas quebra na execução se todas as lacunas do contrato não forem preenchidas.



# Construindo sua própria ABC

```
from abc import ABCMeta, abstractmethod
```

```
class Shape(metaclass=ABCMeta):  
    def __init__(self, id):  
        self._id = id
```

```
@abstractmethod  
def display(self):  
    pass
```

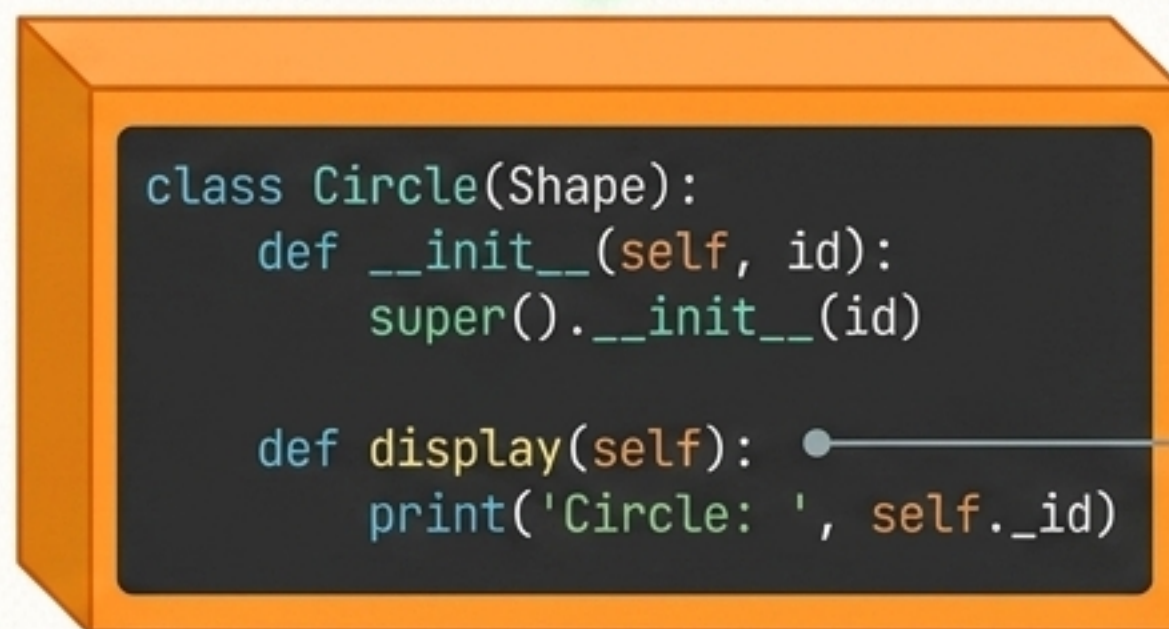
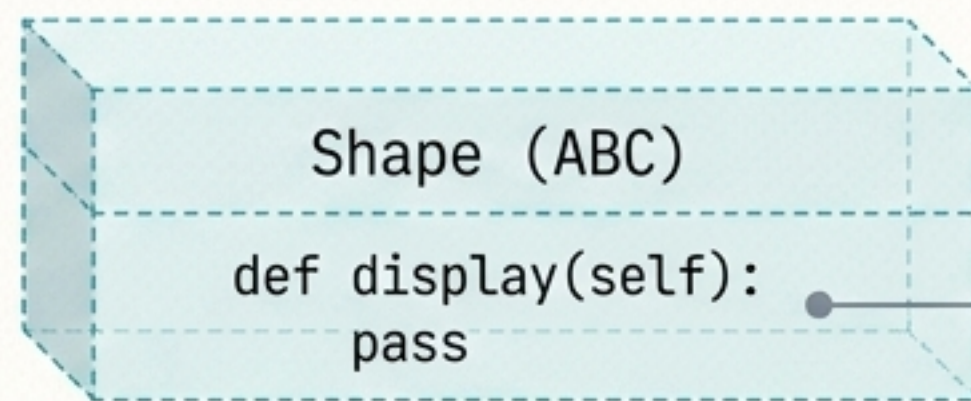
**A Fundação:** O módulo abc fornece a infraestrutura necessária.

**O Motor:** É o que transforma uma classe comum em uma base fundamentalmente abstrata.

**A Regra:** Este decorator sinaliza que qualquer subclasse deve obrigatoriamente fornecer sua própria implementação para display().



# Cumprindo o Contrato (A Classe Concreta)



A lacuna é preenchida com comportamento real.



`c = Circle('circle1')` -> Sucesso! (Instância criada)



# A Ausência do termo Interface



Diferente de Java ou C#, o Python não possui a palavra-chave explícita "interface".

## A Regra Prática

Se uma Abstract Base Class for projetada **APENAS** com **métodos e propriedades abstratas** (sem nenhuma implementação de comportamento concreto ou estado)... ela age **exatamente** como um contrato formal de Interface.

ABC Pura

Interface Tradicional



# Matriz de Diagnóstico Estrutural

|                                         | Classe Normal<br>(Concreta)      | Classe Abstrata<br>(ABC mista) | Interface<br>(ABC pura)           |
|-----------------------------------------|----------------------------------|--------------------------------|-----------------------------------|
| É Instanciável?                         | ✓                                | ✗                              | ✗                                 |
| Pode ter métodos/atributos concretos?   | ✓                                | ✓                              | ✗                                 |
| Pode ter métodos abstratos?             | [✗]<br>(Se tiver, vira abstrata) | ✓                              | [✓]<br>(Exclusivamente abstratos) |
| Força subclasses a implementar métodos? | ✗                                | ✓                              | ✓                                 |



# Subclasses Virtuais: Burlado a Herança Direta

Com ABCs, uma classe não precisa usar herança direta para ser considerada uma subclasse.

O método **register()** formaliza o Duck Typing. Registramos Employee como se fosse um Person no nível da ABC.

```
issubclass(Employee, Person)
```

-> Retorna False

Passo 1: Antes

**Person.register(Employee)**

Passo 2: A Magia

```
issubclass(Employee, Person)
```

-> Agora retorna True!

```
isinstance(e, Person)
```

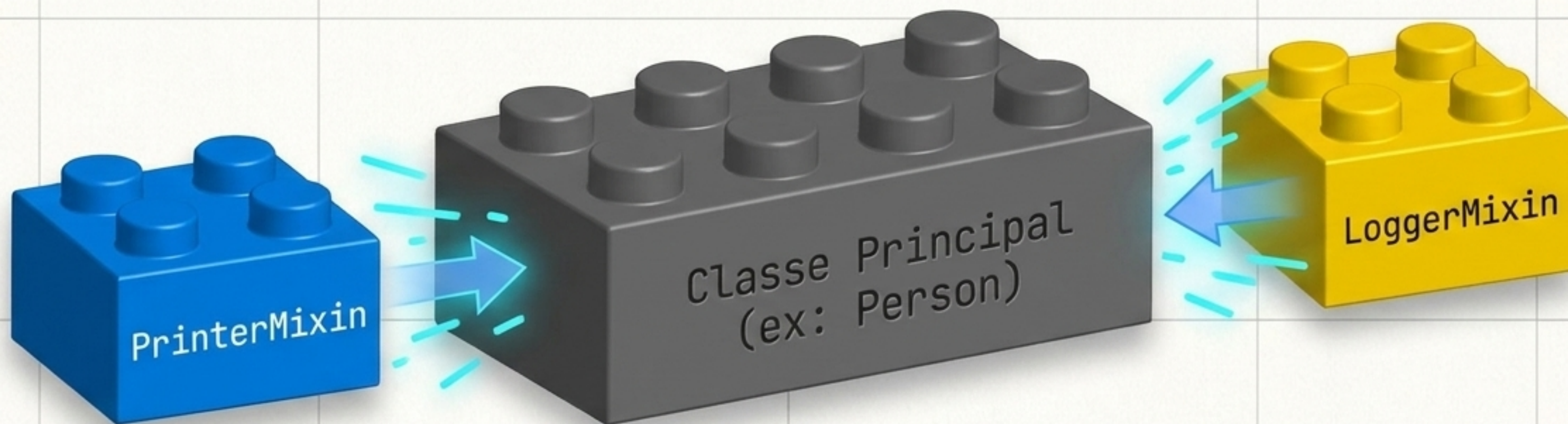
-> True!

Passo 3: Depois

Permite flexibilidade extrema ao adaptar bibliotecas de terceiros sem reescrever hierarquias de classes.



# A Filosofia dos Mixins



Mixins são classes focadas em fornecer funcionalidades secundárias e utilitárias (geralmente concretas) que podem ser misturadas a outras classes usando herança múltipla.

## A Regra de Ouro

Mixins nunca devem ser instanciados sozinhos.  
Eles não existem por si mesmos, apenas para enriquecer outras classes.



# Implementando Mixins na Prática

O PrinterMixin é uma ABC utilitária independente.

```
class PrinterMixin(metaclass=ABCMeta):  
    def print_me(self):  
        print(self)
```

```
class Employee(Person, PrinterMixin):  
    pass # Lógica de Employee aqui...
```

```
e = Employee('Megan', 21, 'MS123')  
e.print_me() # Método herdado do Mixin!
```

A magia da Herança Múltipla. O Employee ganha superpoderes utilitários sem alterar a classe base Person.

O método print\_me() está agora perfeitamente disponível na instância de Employee.



# Mixins e Restrições Implícitas

```
class IDPrinterMixin(metaclass=ABCMeta):  
    def print_id(self):  
        print(self.id)
```



**Atenção:** Este Mixin assume cegamente que a classe hospedeira possui o atributo **.id**



**Cenário de Sucesso:** Misturar com Employee (que tem id) funciona perfeitamente.



**Cenário de Falha:** Misturar com Person (que NÃO tem id) gerará um erro fatal ao chamar o método.

Mixins impõem pré-requisitos invisíveis. Uma documentação rigorosa e design limpo são fundamentais.



# O Ecossistema de Classes em Python (O Big Picture)

## 3. Classes Concretas

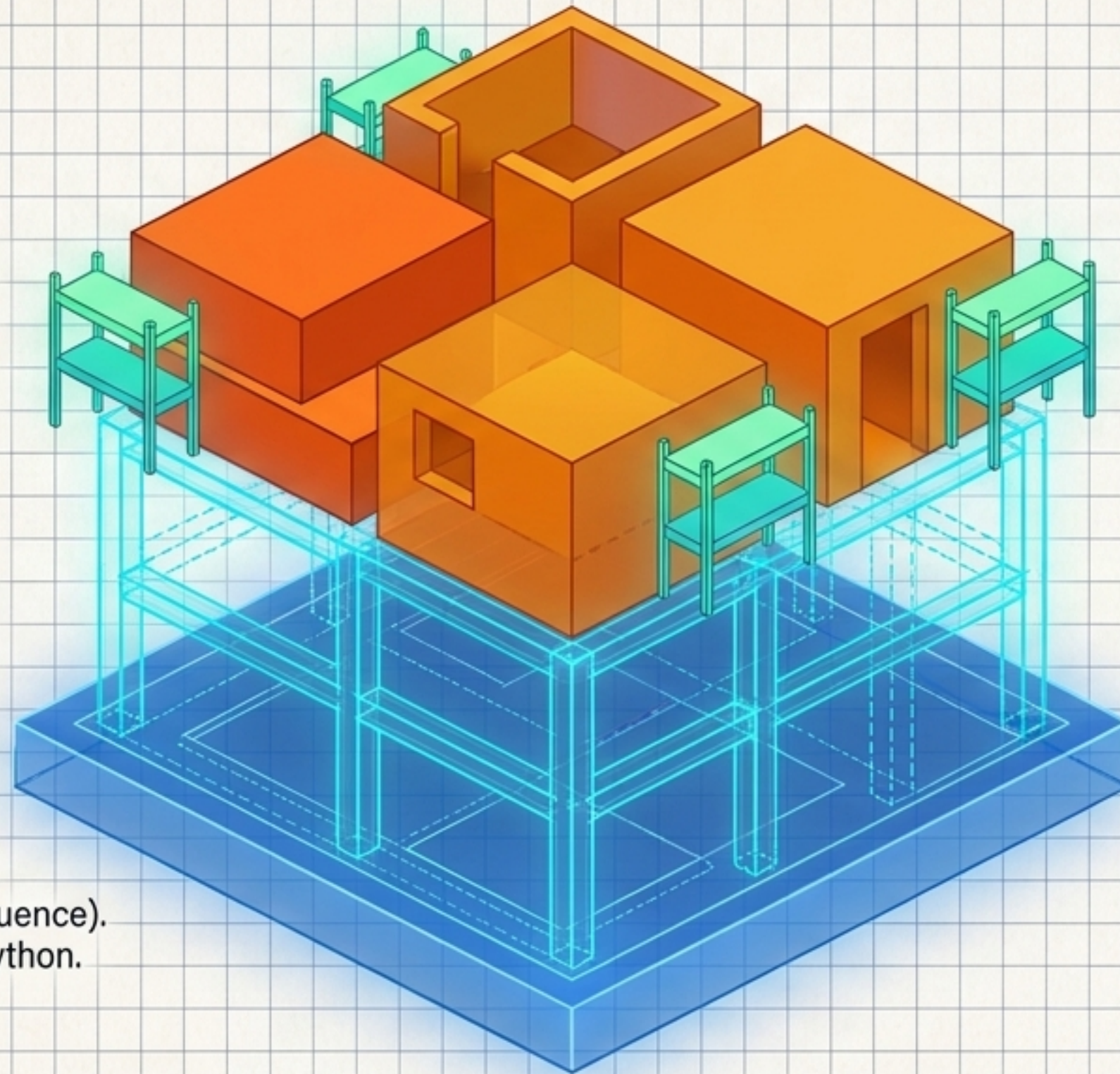
A Execução (ex: Circle, Bag).  
Os objetos reais e instanciáveis que obedecem aos contratos base.

## 2. Custom ABCs

A Estrutura (ex: Shape). Os  
contratos que o arquiteto do projeto  
cria para manter a ordem.

## 1. Built-in ABCs

Fundação (ex: collections.MutableSequence).  
A infraestrutura nativa invisível do Python.



## 4. Mixins

Módulos Anexos. Utilitários plugáveis  
injetados lateralmente via herança  
múltipla para dar superpoderes.



# Resumo da Arquitetura (Takeaways Essenciais)

## 1 Contratos Obrigatórios

ABCs não podem ser instanciadas. Elas servem para forçar as subclasses a implementar métodos fundamentais.

## 2 O Módulo abc

Para criar uma classe abstrata genuína, usa-se a metaclass=ABCMeta e o decorator @abstractmethod.

## 3 Interfaces Nativas

O Python emula interfaces tradicionais através de ABCs compostas exclusivamente por métodos abstratos.

## 4 Subclasses Virtuais

O método register() permite burlar a herança direta de classes, validando tipos por Duck Typing formalizado.

## 5 Mixins são Ferramentas

Utilizam ABCs para encapsular comportamentos utilitários projetados para injeção lateral via herança múltipla.